

Official Report: AWS API Gateway Security Scanner (Lab)

Author: Solomon Mwaura

Date: April 27, 2026

Subject: Development of a Python Script for AWS API Gateway
Security Analysis

1. Introduction

This report details the development of `API_final.py`, a Python script designed to simulate the scanning of AWS API Gateway for security misconfigurations. Developed as part of a lab exercise for a junior security analyst, the script focuses on identifying common vulnerabilities such as missing authentication, weak authentication, insecure HTTP usage, and overly permissive CORS settings. The lab emphasizes a theoretical approach, using mock data to simulate AWS interactions without requiring actual cloud credentials.

2. Lab Instructions and Script Implementation

The script was developed following a step-by-step process as outlined in the lab instructions. Each task contributed to building a functional, albeit simulated, API security scanner.

2.1. Task 1: Define the Scope and Requirements

The script is specifically designed to scan AWS API Gateway. The primary vulnerabilities assessed are:

- Missing authentication mechanisms (e.g., open endpoints).
- Weak authentication (e.g., Basic Auth instead of OAuth2).
- Insecure HTTP usage (unencrypted traffic instead of HTTPS).
- Overly permissive CORS settings.

These requirements are documented within the `check_api_security` function as comments.

2.2. Task 3: Enumerate Web Applications in the Cloud Environment

The `list_apis()` function was modified to return mock API data, simulating the enumeration of API Gateway instances without actual `'boto3'` calls. This allows the script to run in a sandboxed environment.

```
def list_apis():  
    """  
    Simulates an API Gateway response instead of fetching from AWS.  
    This list matches the IDs in our mock configuration dictionary.  
    """  
    mock_response = [
```

```

        {'id': 'abc123', 'name': 'Public-Finance-API'},
        {'id': 'xyz789', 'name': 'Secure-HR-API'}
    ]
    return mock_response

```

2.3. Task 4: Scan for Misconfigurations and Weak Authentication

The `check_api_security(api_id)` function was enhanced to include mock configuration data and logic to identify the specified vulnerabilities. It checks for ``auth_type``, ``use_https``, and ``cors`` settings.

```

def check_api_security(api_id):
    # ... (docstring and findings initialization)

    mock_api_configurations = {
        "abc123": {"auth_type": "NONE", "use_https": False, "cors": "*"},
        "xyz789": {"auth_type": "COGNITO_AUTH", "use_https": True, "cors": "*"}
    }

    api_config = mock_api_configurations.get(api_id, {})

    print(f"\n Checking API {api_id} for security issues...\n")

    if api_config.get("auth_type") == "NONE":
        warning = "WARNING: API has NO authentication! This API is open to the public."
        print(f"{warning} This API is open to the public.\n")
        findings["issues"].append(warning)
        findings["status"] = "Vulnerable"

    if not api_config.get("use_https"):
        warning = "WARNING: API does not enforce HTTPS! Data might be exposed in transit."
        print(f"{warning} Data might be exposed in transit.\n")
        findings["issues"].append(warning)
        findings["status"] = "Vulnerable"

    if api_config.get("cors") == "*":

```

```

        warning = "WARNING: API has overly permissive CORS settings."
        print(f"{warning} May allow cross-origin attacks.\n")
        findings["issues"].append(warning)
        findings["status"] = "Vulnerable"
    # ... (rest of the function)

```

2.4. Task 5: Test for Security Vulnerabilities (Simulated)

Within `check_api_security(api_id)`, a simulated test for unauthorized requests was added. This demonstrates how an attacker might attempt to access an API endpoint and how the system would respond based on its configuration.

```

    # ... (previous checks)

    # --- Testing API Endpoints (Simulated Unauthorized Requests) ---
    test_url = f"https://{api_id}.execute-api.us-east-1.amazonaws.com/p"
    print(f"Testing API endpoint for {api_id}...")

    if api_config.get("auth_type") == "NONE":
        print(f"RESPONSE: 200 OK - Data accessed successfully! (Vulnerable)")
    else:
        print(f"RESPONSE: 401 Unauthorized - Access Denied. (Authenticated)")

    print("-" * 40)
    return findings

```

2.5. Task 6: Generate and Log Findings

The `check_api_security(api_id)` function now collects findings into a dictionary, which is then appended to a ``scan_report`` list in the ``main`` function. A summary report is generated and printed to the console, detailing the status and any issues found for each scanned API.

```

# ... (inside main function)
scan_report = []
# ... (loop through apis)
for api in apis:
    # ... (call check_api_security)
    result = check_api_security(api['id'])
    scan_report.append(result)

print("\n" + "="*50)
print("          API SECURITY SCAN SUMMARY")
print("="*50 + "\n")

if not scan_report:
    print("No APIs were scanned or no issues found.")
else:
    for api_result in scan_report:
        print(f"API ID: {api_result['api_id']}")
        print(f>Status: {api_result['status']}")
        if api_result['issues']:
            print("  Issues:")
            for issue in api_result['issues']:
                print(f"    - {issue}")
    print("-" * 40)

```

2.6. Task 7: Automate and Schedule Regular Scans

The ``main()`` function was modified to include an infinite loop with a ``time.sleep(300)`` call, simulating automated scheduling for regular scans every 5 minutes. This demonstrates how the script could be deployed for continuous monitoring.

```

if __name__ == "__main__":
    while True:
        print("\n" + "="*50)
        print(" Running API Security Scan...")
        print("="*50 + "\n")

```

```
apis = list_apis()
scan_report = []

if not apis:
    print("No APIs found.")
else:
    for api in apis:
        print(f"Scanning API: {api['name']} (ID: {api['id']})")
        result = check_api_security(api['id'])
        scan_report.append(result)

# ... (summary report generation)

print("\n Scan Cycle Complete.")
print(" Next scan in 5 minutes...\n")

time.sleep(300) # Wait 5 minutes before next scan
```

3. Conclusion

The `API_final.py` script successfully implements the requirements of the lab, providing a simulated AWS API Gateway security scanner. It effectively demonstrates the process of identifying common API misconfigurations and weak authentication mechanisms through mock data and simulated testing. This script serves as a valuable educational tool for junior security analysts to understand the principles of API security and the importance of continuous monitoring in cloud environments.

4. Attachments

- `API_final.py`: The Python script for AWS API Gateway security scanning.